

Processador VLIW

Esse é um relatório de um processador VLIW na arquitetura Sagui de Rabo Longo, da disciplina de Arquitetura de Computadores 2026/1. O trabalho consiste em um processador VLIW no logisim, bem como um código para testes. Alunos: Letícia de Oliveira Santos e Iago Pereira

1. Do processador

a. Componentes:

- 2 ULAs (unidades lógico - aritméticas)
 - Somador, subtrator, incrementador, OR lógico, NOT lógico, shift left register e shift right register
- Control - unidade que lida com os sinais de controle
- Pc_Control - unidade que lida com os sinais de controle do PC
- Move - unidade que faz a operação move
- Zero - unidade que determina se um número é zero, se não for, retorna 1
- Banco de Registradores - onde há escrita e leitura dos registradores
- Dual port Ram - memória de dados e instruções
- is_move: unidade que determina se a instrução é move ou não

b. Tabelas de Controle

i. Lane 1 LD/ST/MOVE

Instr	MemRead	MemWrite	RegWriteLn1	Mux_L1
ld	1	0	1	0
st	0	1	0	0
movh	0	0	1	1
movl	0	0	1	1
outras	0	0	0	0

ii. Lane 2 BR/JUMP

Instr	Sig_Brzt	Sig_Brzi	Sig_JR	Sig_Bnzt
brzt	1	0	0	0
brzi	0	1	0	0
jr	0	0	1	0
bnzt	0	0	0	1
outras	0	0	0	0

iii. Lane 3 e 4 ULA

Instr	RegWriteLn3/4	OpUla
add	1	000
sub	1	001
inc	1	010
or	1	011
not	1	100
slr	1	101
srr	1	110
outras	0	X

L1 (1/4): movh 0
L2 (2/4): nop
L3 (3/4): sub R3, R3 // r3 = 0
L4 (4/4): sub R2, R2 // r2 = 0

01.

L1 (1/4): movl -4 // r0 = 12
L2 (2/4): nop
L3 (3/4): sub R1, R1 // r1 = 0 (r1 = i)
L4 (4/4): nop

02.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R3, R0 // r3 = 12 (contador)
L4 (4/4): nop

03. // > A[i] = i

L1 (1/4): movh 4
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

04.

L1 (1/4): movl 0 r0 = 64 (base do vetor A)
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

05.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R0, R1 // r0 = baseA + i
L4 (4/4): nop

06.

L1 (1/4): st R1, R0 // Grava i na memória, em 64+i
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

07. // > B[i] = i + 20

L1 (1/4): movh 1 //
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

08.

L1 (1/4): movl 4 // r0 = 20
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

09.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R2, R0 // r2 = 20
L4 (4/4): nop

10.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R2, R1 // R2 = 20 + i
L4 (4/4): nop

11.

L1 (1/4): movh 5
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

12.

L1 (1/4): movl 0 //base do vetor b (80)
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

13.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R0, R1 // r0 = endereço do B + i
L4 (4/4): nop

14.

L1 (1/4): st R2, R0 // grava na memória B[i]
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

15.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): sub R2, R2
L4 (4/4): nop

16.

L1 (1/4): movh 6
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

17.

L1 (1/4): movl 0 //base do vetor R (96)
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

18.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R0, R1 // r0 = Base do R + i (endereço)
L4 (4/4): nop

19.

L1 (1/4): st R2, R0 // grava 0 no primeiro endereço de r
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

20.

L1 (1/4): movh 0 //r0 = 0
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

21.

L1 (1/4): movl 1 // R0 = 1
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

22.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): inc R1 // i++ (ULA 1)
L4 (4/4): sub R3, R0 // R3-- (ULA 2)

23.

L1 (1/4): movh 0
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

24.

L1 (1/4): movl 3 // Alvo do pulo (03)
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

25.

L1 (1/4): nop
L2 (2/4): bnzr R3, R0 // Salta se R3 != 0
L3 (3/4): nop
L4 (4/4): nop

//reseta contadores

26.

L1 (1/4): movh 0
L2 (2/4): nop
L3 (3/4): sub R1, R1 // i = 0
L4 (4/4): nop

27.

L1 (1/4): movl -4 // R0 = 12
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

28.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R3, R0 // R3 = 12 de novo
L4 (4/4): nop

//soma dos vetores

29. // > Ler A[i]

L1 (1/4): movh 4 // Base A
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

30.

L1 (1/4): movl 0
L2 (2/4): nop
L3 (3/4): nop
L4 (4/4): nop

31.

L1 (1/4): nop
L2 (2/4): nop
L3 (3/4): add R0, R1 // Endereço A[i]

L4 (4/4): nop

32.

L1 (1/4): ld R2, R0 // R2 recebe A[i]

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

33. // > Ler B[i]

L1 (1/4): movh 5 // Base B

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

34.

L1 (1/4): movl 0

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

35.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R0, R1 // Endereço B[i]

L4 (4/4): nop

36.

L1 (1/4): ld R0, R0 // R0 recebe B[i]

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

37. // > Fazer a Soma

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R2, R0 // R2 = A[i] + B[i]

L4 (4/4): nop

38. // > Gravar em R[i]

L1 (1/4): movh 6 // Base R

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

39.

L1 (1/4): movl 0

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

40.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R0, R1 // Endereço R[i]

L4 (4/4): nop

41.

L1 (1/4): st R2, R0 // Grava resultado na RAM

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

42. // > Controle do Laço

L1 (1/4): movh 0

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

43.

L1 (1/4): movl 1 // R0 = 1

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

44.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): inc R1 // i++ (ULA 1)

L4 (4/4): sub R3, R0 // R3-- (ULA 2)

45.

L1 (1/4): movh 1

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

46.

L1 (1/4): movl -3 // Alvo do pulo (29 = 0x1D)

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

47.

L1 (1/4): nop

L2 (2/4): bnzr R3, R0 // Salta se R3 != 0

L3 (3/4): nop

L4 (4/4): nop

48.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): sub R0, R0 // Garante R0 = 0

L4 (4/4): nop

49.

L1 (1/4): nop

L2 (2/4): brzi 0 // Pula para a própria linha

L3 (3/4): nop

L4 (4/4): nop

Relatório do Desenvolvimento do Trabalho 3

Datapath do processador:

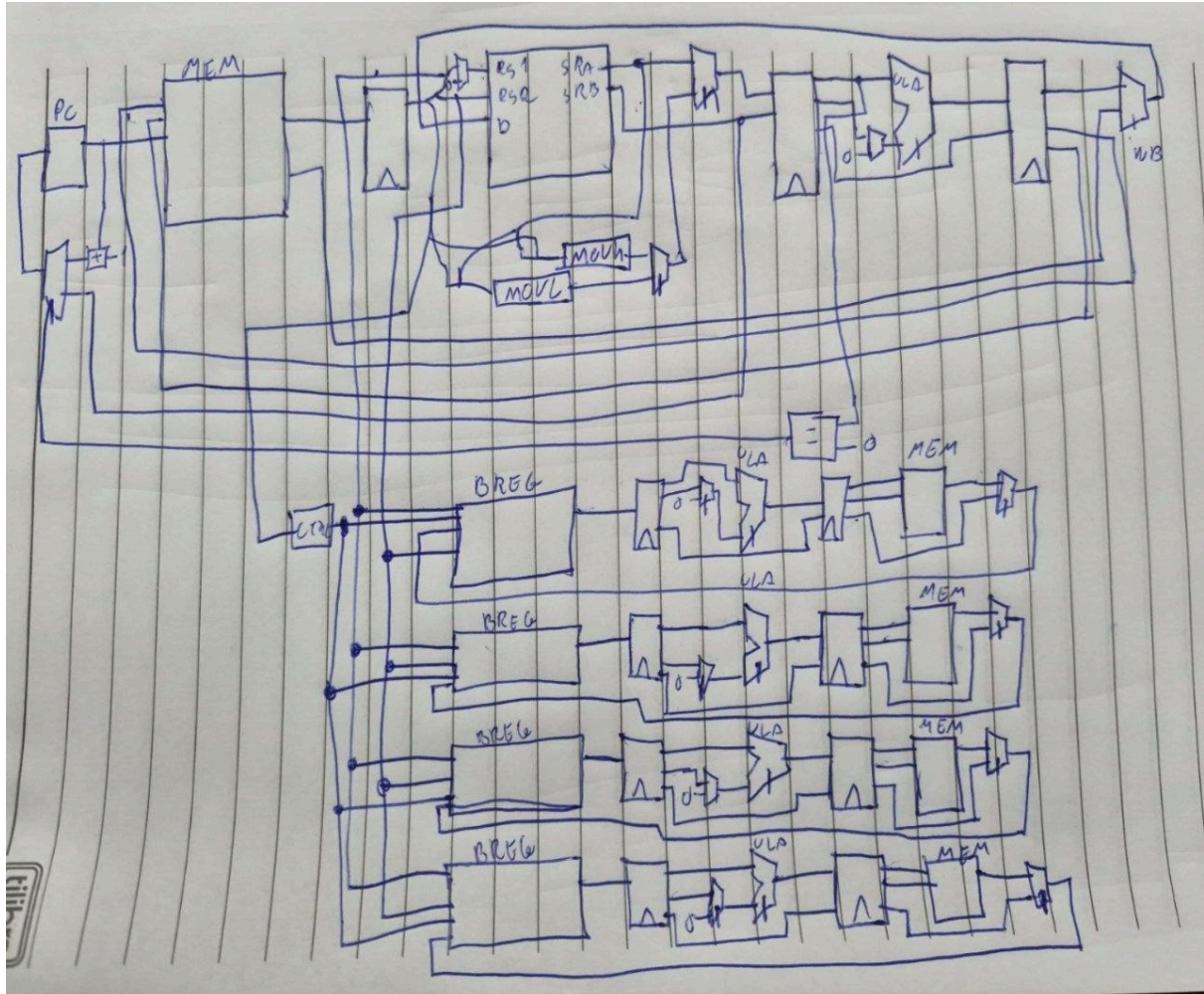


Tabela de códigos da ULA escalar:

opcode	instrução
00	s.add
01	s.sub
10	s.and

Tabela de códigos da ULA vetorial:

opcode	instrução
00	v.add
01	v.sub
10	v.xor
11	v.inc4

Tabela de controle:

	reg_a	sel_im m	wbreg	wm	wb	is_mov	opula_ escalar	is_brzr	mux_mo v
s.ld	0	0	1	0	1	0	00	0	0
s.st	0	0	0	1	0	0	00	0	0
s.movh	1	0	1	0	0	1	00	0	1
s.movl	1	1	1	0	0	1	00	0	1
s.add	0	0	1	0	0	0	00	0	0
s.sub	0	0	1	0	0	0	01	0	0
s.and	0	0	1	0	0	0	10	0	0
s.brzr	0	0	0	0	0	0	00	1	0
v.ld	0	0	1	0	1	0	00	0	0
v.st	0	0	0	1	0	0	00	0	0
v.movh	1	0	1	0	0	1	00	0	1
v.movl	1	1	1	0	0	1	00	0	1
v.add	0	0	1	0	0	0	00	0	0
v.sub	0	0	1	0	0	0	01	0	0
v.xor	0	0	1	0	0	0	10	0	0
v.inc4	0	0	1	0	0	0	11	0	0

Instruções implementadas:

Eu implementei uma instrução de xor vetorial (v.xor) e uma instrução que incrementa 4 no vetorial também (v.inc4);

V.XOR:

Implementei essa instrução, pois o xor tem uma usabilidade de zerar o registrador de forma prática e também tem como funcionalidade manipular bits, invertendo os bits, por exemplo.

Opcode	Tipo	Mnemonic	Nome	Operação
1110	R	v.xor	xor	$VR[ra] = VR[ra] \wedge VR[rb]$

V.INC4:

Optei por essa instrução, pois ela faz algo que sempre acontece no vetorial de passar para o endereço 4 vezes posterior em cada iteração do loop, dessa forma não é necessário fazer o v.movl, economizando registradores.

Opcode	Tipo	Mnemonic	Nome	Operação
1111	R	v.inc4	incrementa 4	$VR[ra] = VR[ra] + 4$

Programa em assembly:

```
s.movl 3 ; r1 recebe 3
s.sub r2, r2 ; r2 recebe 0
s.sub r3, r3 ; r3 recebe 0
v.xor r2, r2 ; r2 recebe 0 (ponteiro para o Vetor inicial)
s.add r2, r1 ; r2 recebe 3
s.movl 1 ; r1 recebe 1
s.add r0, r0 ; nop
s.add r0, r0 ; nop
s.add r3, r1 ; r3 recebe 1
```

loop_preenche_vetores:

```
v.xor r3, r3 ; r3 recebe 0
v.movh 1 ; r1 recebe 16
```

s.add r0, r0 ;
v.add r3, r2 ; r3 recebe ponteiro para o Vetor
v.movl 4 ; r1 recebe 20
s.add r0, r0 ;
v.add r3, r0 ; r3 recebe ponteiro para o Vetor + ID
v.add r1, r2 ; r1 recebe 20 + ponteiro para o Vetor
s.add r0, r0 ;
v.st r3, r2 ; M[r2] guarda o Vetor A
v.add r1, r0 ; r1 recebe 20 + ponteiro para o Vetor + ID
v.xor r3, r3 ; r3 recebe 0
s.add r0, r0 ;
s.add r0, r0 ;
v.add r3, r2 ; r3 recebe ponteiro para o Vetor
s.add r0, r0 ;
s.add r0, r0 ;
v.inc4 r3, r3 ; r3 recebe ponteiro para o Vetor + 4
s.add r0, r0 ;
s.add r0, r0 ;
v.inc4 r3, r3 ; r3 recebe ponteiro para o Vetor + 8
s.add r0, r0 ;
s.add r0, r0 ;
v.inc4 r3, r3 ; r3 recebe ponteiro para o Vetor + 12 (Vetor B)
s.add r0, r0 ;
s.add r0, r0 ;
v.st r1, r3 ; M[r3] guarda o Vetor B
v.movh 1 ; r1 16
v.xor r3, r3 ; r3 recebe 0
s.add r0, r0 ;
v.movl 8 ; r1 recebe 24
s.add r0, r0 ;
s.add r0, r0 ;
v.add r1, r2 ; r1 recebe 24 + ponteiro para o Vetor (Endereço R)
v.inc4 r2, r2 ; ponteiro para o Vetor +recebe 4
s.sub r2, r3 ; r2 subtrai 1
v.st r3, r1 ; M[r1] guarda o Vetor R
s.movh 4 ; prepara a soma
s.add r0, r0 ;
s.add r0, r0 ;
s.movl 0 ; r1 recebe 64
s.add r0, r0 ;

```
s.add r0, r0 ;  
s.brzr r2, r1 ; Pula para fora do loop  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.movh 0 ; Prepara loop_preenche_vetores  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.movl 9 ; r1 recebe 9  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.brzr r0, r1 ; volta para o inicio do loop  
s.add r0, r0 ;  
s.add r0, r0 ;
```

prepara_soma:

```
s.movh 0 ;  
s.sub r2, r2 ; Zera r2  
v.xor r2, r2 ; Zera ponteiro para o Vetor r2  
s.add r0, r0 ;  
s.movl 3 ; r1 recebe 3  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.add r2, r1 ; r2 recebe 3
```

loop_soma:

```
v.xor r3, r3 ; r3 recebe 0  
s.add r0, r0 ;  
s.add r0, r0 ;  
v.add r3, r2 ; r3 recebe ponteiro para o Vetor  
s.add r0, r0 ;  
s.add r0, r0 ;  
v.ld r1, r3 ; r1 recebe lê Vetor A  
v.xor r3, r3 ; r3 recebe 0  
s.add r0, r0 ;  
s.add r0, r0 ;  
v.add r3, r2 ; r3 recebe ponteiro para o Vetor  
s.add r0, r0 ;  
s.add r0, r0 ;
```

```

v.inc4 r3, r3 ; +4
s.add r0, r0 ;
s.add r0, r0 ;
v.inc4 r3, r3 ; +8
s.add r0, r0 ;
s.add r0, r0 ;
v.inc4 r3, r3 ; +12 (Endereço B)
s.add r0, r0 ;
s.add r0, r0 ;
v.ld r3, r3 ; r3 recebe Lê Vetor B
s.add r0, r0 ;
s.add r0, r0 ;
v.add r3, r1 ; r3 recebe B + A
v.movh 1 ; r1 recebe 16
s.add r0, r0 ;
s.add r0, r0 ;
v.movl 8 ; r1 recebe 24
s.add r0, r0 ;
s.add r0, r0 ;
v.add r1, r2 ; r1 recebe 24 + ponteiro para o Vetor
v.inc4 r2, r2 ; ponteiro para o Vetor recebe +4
s.sub r2, r3 ; r2 subtrai 1
v.st r3, r1 ; M[r1] guarda o Vetor R

```

```

s.movh 7 ; Prepara para o fim_do_programa
s.add r0, r0 ;
s.add r0, r0 ;
s.movl 14 ; r1 recebe 126
s.add r0, r0 ;
s.add r0, r0 ;
s.brzr r2, r1 ; pula para fora do loop
s.add r0, r0 ;
s.add r0, r0 ;
s.movh 4 ; Prepara loop_soma
s.add r0, r0 ;
s.add r0, r0 ;
s.movl 8 ; r1 recebe 72
s.add r0, r0 ;
s.add r0, r0 ;
s.brzr r0, r1 ; volta para o inicio do loop

```



```
s.add r0, r0 ;  
s.add r0, r0 ;
```

fim_do_programa:

```
s.movh 7 ; 126  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.movl 14 ; r1 recebe 126  
s.add r0, r0 ;  
s.add r0, r0 ;  
s.brzr r0, r1 ; salta pro mesmo endereço sempre  
s.add r0, r0 ;  
s.add r0, r0 ;
```